Ders 7 Çalışma Özeti

Ders 7 Çalışma Özeti

Genel Konular

- Refactoring (yeniden yapılandırma) nedir?
  - Bir kod parçasının ne yaptığını değiştirmeden nasıl yaptığını değiştirmektir. Başka bir deyişle, dışarıya sunulan arayüz değişmeden iç yapının değiştirilmesidir.
  - Amaç, kodun iç kalite özelliklerini iyileştirmektir. Hoca kaliteyi en basit biçimiyle ikiye ayırır: iç kalite özellikleri yazılımı geliştirenleri, dış kalite özellikleri yazılımı kullananları ilgilendirir.
  - İç kaliteye örnekler: taşınabilirlik (az değişiklikle başka platforma taşınabilme), bağlaşım (coupling) ve uyum (cohesion), anlaşılabilirlik. Dış kaliteye örnekler: verimlilik (sistem kaynaklarını az kullanmak), kullanım kolaylığı. İç kalitedeki iyileşmenin dış kaliteye de olumlu etkisi olabilir, ama hedef iç kalitedir.
  - Refactoring ile koda esneklik ve yeniden kullanılabilirlik kazandırılır. Bir üyeye daha anlamlı bir ad vermek bile bir refactoring eylemidir ve yararı büyüktür.
  - Refactoring yalnız bakım aşamasında değil, tasarımda ve kodda, her aşamada yapılabilir.
- Refactoring nasıl ve ne zaman yapılır? (ekrandan, 00:10:00)
  - Çevik yaklaşımla, küçük adımlarla: mevcut gereksinimleri karşılayacak ilk tasarım yapılır; ilk tasarımın mükemmel olması ve her bilinmezi öngörmesi gerekmez; yeterince iyi tasarım gerçeklenmeye başlanır; önceki adımlardan birinde yanlış karar verildiği anlaşılınca refactoring yapılır.
  - Gereksinimler, kurallar ve yönetmelikler altı ay gibi kısa sürede bile değişebilir. Hoca "mükemmel, iyinin katilidir" sözünü hatırlattı.
  - Sürekli sınama ile: küçük bir değişiklik bile öngörülmeyen hatalar doğurabilir; her refactoring eyleminden sonra birim sınamaları yeniden çalıştırılmalıdır.
  - Ne zaman: kod derlenip toparlanmaya gerek duyduğunda; yeni özellik eklemekte zorlanmaya başlandığında; bir hatayı bulmak ve düzeltmek için birçok ayrı yeri kurcalamak gerektiğinde. Kısaca "kod kötü kokmaya başladığında". Çevik süreçlerde bir sprint yeni özellik eklemek yerine refactoring'e ayrılabilir.
- Code smell (kod kusuru) (ekrandan, 00:20:00)
  - Yazılan kodu iyileştirmek gerektiğinin işaretleridir; kodda karşılaşılan terslikler, esnek olmayan noktalardır.
  - Hoca terimi "kod kokusu" yerine "kod kusuru" diye çevirir: code smell bir hata (bug) değildir, yalnız esnek olmayan ya da iç kalitesi düşük kod ortaya çıkarır. "Güzel bir insandır ama ufak bir kusuru vardır" benzetmesi.
  - Kullanılan programlama yaklaşımının kusurlu kullanımından doğar. En temel örnek: nesneye yönelik tasarımda yapısal programlama düşünce tarzıyla karar vermek.
  - Zamanla, farklı kişilerin farklı zamanlarda yaptığı değişikliklerle yazılım yamalı bohçaya (ya da spagetti koda) dönüşebilir. Bir değişiklik için çok farklı yerlerde işlem yapmak gerekiyorsa ya da denetim akışı gereksiz karmaşık parçalardan geçiyorsa bu işarettir.
  - Bir refactoring eylemi yeni bir kod kusuru doğurabilir; o durumda yeni refactoring eylemleri yürütülür.
- JUnit ile birim sınaması (ekrandan, 00:30:00)
  - Birim sınaması en küçük yazılım bileşeninin sınanmasıdır; nesneye yönelik programlamada bireysel sınıfların sorumluluklarının sınanmasıdır.
  - Birim testlerini programcılar, varsa ayrı bir sınama ekibi de hazırlar. Testler kodlama-

dan önce (TDD), kodlama sırasında ya da kodlamadan sonra tasarlanabilir. TDD'de bir satır üretim kodu yazmadan önce testler yazılır ve kod yalnız bu testleri geçmeye yönelik yazılır.
  - Sınanan birimin işbirliği yaptığı sınıflar henüz kodlanmamışsa, onların yerine geçen vekil kod yazılır (slaytta: stub, dummy, surrogate, proxy). Vekil, gerçek sınıflar gerçeklenene dek kullanılır; birlikte çalışma tümleştirme sınamalarında denetlenir.
  - xUnit ailesi: X yerine dilin harfi gelir (Java için JUnit, C# için csUnit). Eclipse'te build path → add libraries ile, NetBeans'te hazır gelir; en son sürüm junit.org'dan indirilebilir.
- JUnit 3.x kullanımı
  - Her uygulama sınıfı için bir test sınıfı hazırlanır. Test sınıfı `junit.framework.TestCase`'ten kalıtımla türetilir ve adı `test` ile başlar (örn. `AClass1` için `TestAClass1`).
  - Her test metodu bir test case'e karşılık gelir ve adı `test` ile başlamalıdır; geri kalanı testin amacını anlatan, gerekirse uzun bir addır.
  - Sonuç `Assert` sınıfının statik metotlarıyla denetlenir: `assertEquals(beklenen, gerçekleşen)`, `assertNotEquals`, `assertTrue`, `assertFalse`, `assertNull`, `assertNotNull`. Hepsinin ilk parametre olarak bir açıklama dizgisi alan aşırı yüklenmiş (overloaded) sürümleri vardır; test başarısız olursa bu metin hata iletisine eklenir.
  - `setUp()` korumalı (protected) bir metottur; test öncesi sahne (nesneler, ilişkiler) burada hazırlanır. Çerçeve onu her test metodundan önce otomatik olarak yeniden çalıştırır.
  - Test sınıfları bir `suite()` metoduyla test takımında (TestSuite) toplanabilir.
- JUnit 4.x yenilikleri (ekrandan, 00:40:00)
  - Geriye doğru uyumludur; ek olarak annotation desteği getirir (büyük/küçük harf duyarlıdır).
  - `TestCase`'ten kalıtım gerekmez; `import static org.junit.Assert.*;` ve `import org.junit.*;` eklenir.
  - Sınıf ve metot adlarının `test` ile başlaması gerekmez; metodun başına `@Test` konur. `setUp` yerine adı ne olursa olsun `@Before` ile işaretlenmiş metot kullanılır.
  - Atılması gereken istisnaların sınanması: `@Test(expected=SomeException.class)`. İstisna atılırsa test başarılı sayılır, atılmazsa başarısız sayılır; her iki durumda yürütme sonraki testle sürer.
- Fowler'ın video kaset kiralama örneği (ekrandan, 00:50:00, 00:53:10)
  - Kod kusurları ve refactoring konusu Martin Fowler'ın "Refactoring" kitabından alınmıştır. Kitaptaki bütün örnekler bu problem alanındadır.
  - Sınıflar: `Customer` (`_name`, `Vector<Rental> _rentals`, `addRental`, `getName`, `statement`), `Rental` (film ve kiralanan gün sayısı, getter'lar), `Movie` (ad, fiyat kodu; `REGULAR = 0`, `NEW_RELEASE = 1`, `CHILDRENS = 2` public final static alanlar; fiyat kodu için setter da vardır, çünkü yeni film zamanla normal filme döner ya da yanlış işaretlenmiş olabilir). Bir müşterinin çok kira kaydı, bir kira kaydının bir filmi vardır.
  - Kodda alt çizgi ile başlayan adlar üye alandır (Fowler'ın gösterimi).
  - `statement()` makbuz metnini üretir. Ücret kuralları: normal film 2 + (gün − 2) × 1,5 (2 günden uzunsa); yeni film gün × 3; çocuk filmi 1,5 + (gün − 3) × 1,5 (3 günden uzunsa). Her kiralama 1 puan; yeni film 2 gün ya da daha uzun kiralanırsa 1 puan daha.
  - Hoca, Fowler'ın yalnız metot çağrılarını gösteren sıralama şemasını geri dönüşlerle ayrıntılandırarak kendisi çizdi.
- Örnek test sınıfları (ekrandan, 01:00:00, 01:01:30, 01:10:00)
  - `TestCustomer extends TestCase`: `setUp()` bir müşteri ve dört film (biri normal, biri çocuk, ikisi yeni) oluşturur.
  - Testler: `testGetName`, `testStatementWhenEmpty`, `testStatementWithMoviesLongRent`, `testStatementWithMoviesShortRent`, `testNewReleaseRentalBonus`; ayrıca `TestMovie.testMovie`. `TestAll` ikisini bir suite'te toplar; toplam 6 test.
  - Yeşil çubuk: bütün testler geçti. Kırmızı çubuk: bir test geçemedi; JUnit beklenen ve elde edilen dizgilerin ayrıştığı yeri köşeli parantezle gösterir.
- Code smell: Duplicated Code (yinelenen kod) (ekrandan, 01:30:00)
  - Aynı ya da çok benzer kodun birden fazla yerde tekrarlanmasıdır; genellikle kopyala-yapıştırdan doğar. İşlevsellik değişince her kopyada aynı değişiklik yapılmalıdır; sonuç bir bakım kabusudur.
  - Çözümler: Extract Method; kardeş/kuzen sınıflardaysa Pull Up Method; kod aynı de-

ğil benzerse Form Template Method; ilişkisiz sınıflardaysa metot bir sınıfa (ya da yeni bir sınıfa) atanır, diğerleri onu kullanır (ekrandan, 02:00:00).
- Refactoring: Extract Method (metot çıkartma) (ekrandan, 01:40:00, 01:50:00)
  - Bir metodun içindeki kodun bir kısmını yeni bir metot olarak belirlemektir. Birçok kod kusurunu düzeltir.
  - Yeni metoda amacını açıkça belirten isabetli bir ad verilir; böylece alt düzey ayrıntıya girmeden kod anlaşılır ve yorum yazmaya bile gerek kalmaz.
  - Örnek: `printOwing(double amount)` içindeki ayrıntı yazdırma satırları `printDetails(double amount)` metoduna çıkarılır; dışarıya sunulan imza değişmez.
- Code smell: Long Method (aşırı uzun metot)
  - Metot uzadıkça anlaşılması zorlaşır, gereksinim değişikliklerinden etkilenme olasılığı artar, uyumu düşer.
  - Çözüm: kendi içinde uyumlu kod öbekleri bulunur ve her biri ayrı metoda çıkarılır. Diğer çözümler: Decompose Conditional, Replace Temp with Query, Replace Method with Method Object.
  - Örnek (ekrandan, 01:50:00): uzun `printOwing()` üç parçaya ayrılır: `printBanner()`, `getOutstanding()` (borç toplamı, for-each döngüsüyle) ve `printDetails(double outstanding)`. Yeni hâli `printBanner(); printDetails(getOutstanding());`.
- Refactoring: Pull Up Method (metodu yukarı çekme) (ekrandan, 02:02:50)
  - Kardeş sınıflarda aynı gövdeli metot varsa ortak üst sınıfa çekilir (örn. `Salesman` ve `Engineer`'daki `getName`, `Employee`'ye).
  - Gövdeler çok farklıysa metot üst sınıfta soyut tanımlanır (örn. `Customer.chargeFor(start, end)`), böylece tür dönüşümü yapmadan çağrılabilir.
  - Kısmen ortak, kısmen farklıysa Template Method kalıbı uygulanır.
- Refactoring: Decompose Conditional (karar verme komutunu parçalama) (ekrandan, 02:21:00)
  - Aşırı karmaşık `if` koşulları sadeleştirilir. Örnek: `if (date.before(SUMMER_START) || date.after(SUMMER_END))` → `if (notSummer(date));` dallar `winterCharge(quantity)` ve `summerCharge(quantity)` metotlarına çıkarılır.
- Code smell: Long Parameter List (uzun parametre listesi) (ekrandan, 02:30:00)
  - Bir metodun çok parametre alması, hatta bu parametrelerin bazılarının başka metotlarda da birlikte geçmesidir. Bu parametreler birlikte bir varlığın durum bilgisinin bir kısmını ya da tamamını oluşturur.
  - Çözümler: Introduce Parameter Object ve Preserve Whole Object.
- Refactoring: Introduce Parameter Object (parametre nesnesi oluşturma) (ekrandan, 02:40:00)
  - `amountInvoicedIn(start: Date, end: Date)` gibi metotlar `amountInvoicedIn(DateRange)` olur. `DateRange` yalnız kayıt yapısı kalmaz; `isWithinRange(Date)` gibi metotlar da alır, böylece Decompose Conditional ya da Extract Method de yapılmış olur.
  - Zayıf yönü: metot imzası değişir. Geriye doğru uyumluluk şartsa yapılmaz.
- Refactoring: Preserve Whole Object (tüm nesneyi gönder) (ekrandan, 02:37:10)
  - `plan.withinRange(low, high)` yerine `plan.withinRange(daysTempRange())`. Geçici değişkenler de ortadan kalkar.
  - Zayıf yönleri: imza değişir; parametre nesnesinin sınıfıyla metodun sahibi arasında bağımlılık doğabilir. Sınıflar arasında zaten ilişki yoksa ve bu sakıncalıysa yapılmaz.
- Refactoring: Replace Temp with Query (geçici değişken yerine erişim metodu) (ekrandan, 02:41:30, 02:50:00)
  - Bir ifadeyi geçici değişkende saklamak yerine ifadeden metot çıkarılır. Örnek: `getPrice()` → `basePrice() * discountFactor()`.
  - Geçici değişkenler metotları uzatmayı teşvik eder (Long Method) ve kopyala-yapıştıra yol açar; sorgu metodu ise tek yerde düzeltilir.
  - Tartışma: geçici değişken çok yerde kullanılıyorsa (örn. döngü içinde `item.getCharge()` iki kez) hesap tekrarlanır ve başarım düşebilir; hesap uzak bir sunucuya çağrı gerektiriyorsa yapılmaz. Önerilen yol: yapın, profiler ile ölçün, darboğaz oluşursa geri alın.
- Refactoring: Replace Method with Method Object (metot yerine yeni türden nesne) (ekrandan, 02:57:00)
  - Metot uzun ve yerel değişkenleri karmaşık kullanıyorsa, Extract Method yapılamıyorsa, metot yeni bir sınıfa taşınır, yerel değişkenler o sınıfın üyeleri olur. Örnek:

`Order.price()` → `return new PriceCalculator(this).compute();`.

Hocanın Özellikle Vurguladığı Kısımlar

- Kod kusurlarının ve refactoring eylemlerinin özgün (İngilizce) adlarını hatırlamak
  - Hoca terimleri Türkçeye çevirse de literatürdeki özgün adların kullanılacağını ve bilinmesi gerektiğini birkaç kez söyledi.
- Refactoring'in tanımı: dış arayüz değişmez
  - Extract Method örneğinde `printOwing(double amount)` imzası korunur. Introduce Parameter Object ve Preserve Whole Object imzayı değiştirdiği için bu ilkeye karşı zayıf noktadır; hoca bunu tartışılması gereken nokta olarak işaretledi.
- Her refactoring adımından sonra birim testleri yeniden çalıştırılmalı
  - Küçük bir değişiklik bile öngörülmeyen hata doğurabilir.
- Test metotlarına uzun ama açık adlar vermek
  - Hata uygulama kodunda değil test kodunda da olabilir; testin ne yaptığını anlatan ad bunu bulmayı kolaylaştırır.
- Basit görünen testleri küçümsememek
  - `testGetName` aynı zamanda Türkçe karakterlerin doğru saklanıp geri alınıp alınmadığını da sınar.
- Tür dönüşümü (typecasting) bir uyarı işaretidir
  - Bir yerde tür dönüşümü görülüyorsa büyük olasılıkla daha iyi bir tasarım (çoğu kez çok biçimlilik) yapılabilirdi. Extract Method örneğinde for-each döngüsü `(Order)` dönüşümünü kaldırdı (KISS ilkesi).
- if-else zincirleri ve switch-case daha iyi tasarım gerektiğinin alarm zilidir
  - Bunlar, çok sayıda döngü ve yoğun geçici değişken kullanımı kodu uzatma eğilimindedir.
- Küçük metot ve sınıfların başarıma etkisi önemsizdir
  - Metot çağrısının ek yükü vardır, ama modern derleyiciler bunu büyük oranda gidermektedir; bakım kazancı daha büyüktür. İstisna: hesap başka bir sunucuya çağrı gerektiriyorsa.
- Replace Method with Method Object son çaredir
  - Diğer, daha hafif refactoring eylemleri işe yaramadığında kullanılır.

Kısa Tekrar Notları

- Refactoring: ne yapıldığı değil nasıl yapıldığı değişir; dış arayüz korunur; hedef iç kalitedir.
- Çevik ve küçük adımlarla, sürekli sınama ile yapılır; kod kötü kokmaya başlayınca yapılır.
- Code smell bir hata değil, kusurdur; refactoring yeni kusur doğurabilir.
- Birim sınaması: en küçük bileşen; NYP'de sınıf. Eksik işbirlikçiler için vekil kod (stub, dummy, surrogate, proxy).
- JUnit 3: `TestCase`'ten türet, sınıf ve metot adları `test` ile başlasın, `setUp()` her testten önce çalışır, `assertX` metotları, `suite()`.
- JUnit 4: `@Test`, `@Before`, `@Test(expected=...class)`, `import static org.junit.Assert.*`.
- Duplicated Code → Extract Method, Pull Up Method, Form Template Method, ortak sınıfa taşıma.
- Long Method → Extract Method, Decompose Conditional, Replace Temp with Query, Replace Method with Method Object.
- Long Parameter List → Introduce Parameter Object, Preserve Whole Object (imza değişir; Preserve Whole Object bağımlılığı artırabilir).
- Replace Temp with Query: başarım kaybı genelde önemsiz; profiler ile ölç, darboğaz varsa geri al.
- Adlarda "up/down" kalıtım hiyerarşisinde yukarı/aşağı hareket demektir.
- Extract Method, Pull Up Method ve Form Template Method özünde aynı işi yapar: çıkarılan kod bloğu için en uygun yeri arar.

Detaylı Açıklamalar

Refactoring ve kalite

Refactoring, bir kod parçasının dışarıdan görünen davranışını koruyarak iç yapısını değiştirmektir. Hocanın tanımıyla, "ne yaptığını değiştirmeden nasıl yaptığını değiştirmek". Amaç iç kalite özellikleridir: anlaşılabilirlik, taşınabilirlik, bağlaşım ve uyum, esneklik, yeniden kullanılabilirlik. Hoca kalite için pek çok sınıflandırma olduğunu, kendisinin en basitini, yani iç/dış ayrımını tercih ettiğini söyledi. İç kalite geliştiricileri ilgilendirir (örneğin bir yazılımı başka platforma taşımak geliştiricinin işidir); dış kalite kullanıcıları ilgilendirir (örneğin daha az kaynak kullanan kod düşük kapasiteli sistemlerde de çalışır).

Refactoring çevik bir anlayışla yapılır. İlk tasarım her bilinmezi öngörmeye çalışmaz; bugünkü gereksinim kümesini en iyi biçimde modeller. Gereksinimler değiştikçe ya da önceki bir tasarım kararının yanlış olduğu anlaşıldıkça küçük adımlarla refactoring yapılır. Her adımdan sonra birim testleri yeniden çalıştırılır. Refactoring ihtiyacının en önemli işareti, yeni özellik eklemenin ya da hata düzeltmenin giderek daha uzun sürmesidir.

JUnit ile birim sınaması

JUnit 3.x'te test sınıfları `junit.framework.TestCase`'ten türetilir. Hem sınıf hem metot adları `test` ile başlar. `setUp()` her test metodundan önce çerçeve tarafından çağrılır; test metotları sonucu `assertEquals`, `assertTrue` gibi statik metotlarla denetler. `assertEquals(beklenen, gerçekleşen)` en temel kara kutu sınamasıdır: beklenen doğru çıktı ile elde edilen sonuç karşılaştırılır. Diğer varyasyonlar (`assertNull` gibi) aslında `assertEquals` ile de yazılabilir, ama çerçeve işi kolaylaştırmak için hazır sunar.

JUnit 4 geriye doğru uyumludur ve annotation'larla çalışmayı getirir (ekrandan, 00:40:00):

```
import static org.junit.Assert.*;
import org.junit.*;

@Before
public void setUp() { /*Preparations*/ }
@Test
public void testSomething() { /*Do test*/ }

@Test(expected=SomeException.class)
public void testTheException() throws SomeException {
    doSomethingThatCreatesTheException();
}
```

Hoca, ders örneklerinde istisna kullanılmadığı için 3.x sürümüyle devam etti.

Kiralama örneği ve testleri

`statement()` metodunun ücret bölümü (ekrandan, 00:53:10):

```
switch (each.getMovie().getPriceCode()) {
case Movie.REGULAR:
    thisAmount += 2.0;
    if (each.getDaysRented() > 2)
        thisAmount += (each.getDaysRented() - 2) * 1.5;
    break;
case Movie.NEW_RELEASE:
    thisAmount += each.getDaysRented() * 3.0;
    break;
case Movie.CHILDRENS:
    thisAmount += 1.5;
    if (each.getDaysRented() > 3)
        thisAmount += (each.getDaysRented() - 3) * 1.5;
    break;
}
```

```
// add frequent renter points
frequentRenterPoints ++;
// add bonus for a two day new release rental
```

`testStatementWithMoviesLongRent` (ekrandan, 01:01:30): normal film 3 gün, çocuk filmi 4 gün, yeni film 2 gün kiralanır. Beklenen: The Matrix 2 + 1 × 1,5 = 3,5; Monsters, Inc. 1,5 + 1 × 1,5 = 3,0; Surrogates 2 × 3 = 6,0; toplam 12,5; puan 3 + 1 (yeni filmin 2 günlük bonusu) = 4. `testNewReleaseRentalBonus`: yeni film 2 gün (2 puan) ve normal film 1 gün (1 puan), toplam 3 puan.

Hoca bir hatayı canlandırmak için normal filmin günlük ek ücretini 1,5 yerine 2,5 yazdı. Test kırmızı çubuk verdi; JUnit The Matrix satırında beklenen 3,5 yerine 4,5 (2 + 1 × 2,5) bulunduğunu gösterdi. Kod düzeltilince çubuk yeşile döndü.

Extract Method ve Long Method

Long Method örneğinde (ekrandan, 01:50:00) yorumlarla ayrılmış üç öbek vardır: banner yazdırma, borç hesabı ve ayrıntı yazdırma. Sonuç:

```
void printOwing() {
    printBanner();
    printDetails(getOutstanding());
}
double getOutstanding() {
    double outstanding = 0.0;
    for (Order each : _orders)
        outstanding += each.getAmount();
    /* for-each ile döngü kısaldı! */
    return outstanding;
}
void printDetails(double outstanding) {
    System.out.println("name:" + _name);
    System.out.println("amount" + outstanding);
}
```

Önceki hâlde `Enumeration` ve `(Order) e.nextElement()` dönüşümü vardı; jenerik yapı ve JDK 1.5 ile gelen for-each döngüsü dönüşümü kaldırdı. İç içe çağrıda önce `getOutstanding()` çalışır, sonucu `printDetails`'e verilir. Parça sayısının artması bir dezavantaj gibi görünse de değişiklik yapılacak yer, metot adlarına bakılarak hemen bulunur.

Replace Temp with Query

```
/*** ÖNCE ***/
double getPrice() {
    int basePrice = _quantity * _itemPrice;
    double discountFactor;
    if (basePrice > 1000) discountFactor = 0.95;
    else discountFactor = 0.98;
    return basePrice * discountFactor;
}
/*** SONRA ***/
double getPrice() { return basePrice() * discountFactor(); }
private int basePrice() { return _quantity * _itemPrice; }
private double discountFactor() {
    if (basePrice() > 1000) return 0.95;
    else return 0.98;
}
```

(ekrandan, 02:41:30) Sonraki hâlde çarpma iki kez yapılır. Hoca bu fazladan işlemin insanın fark edemeyeceği kadar küçük olduğunu, bakım kazancının yanında göz ardı edilebileceğini söyledi.

- Ders sonrası not: Hoca metot çağrısında çağıranın durum bilgisinin “heap bellekte bir yığına” atıldığını söyledi (01:54:29). Doğrusu, bu bilgi çağrı yığınına (stack) konur; heap dinamik nesnelerin bölgesidir.
- Ders sonrası not: Hoca JUnit 3.x’te istisna testinin yapılamadığını söyledi (00:40:22). Büyük olasılıkla doğrusu: JUnit 3’te özel bir annotation yoktur, ama `try { ...; fail(); } catch (SomeException e) { }` kalıbıyla istisna sınaması yazılabilir.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.